



Laboratory Assessment VII

PEC-2 Laboratory: Full Stack Development - BIT26PE05

To integrate the VakilSahay React frontend with the Spring Boot REST API backend

Varad Sushant Deshmukh [123B1F022] TY-IT-A, PCCOE College

AIM:

To integrate the VakilSahay React frontend with the Spring Boot REST API backend to perform all CRUD operations through the user interface, with proper authentication, error handling, and full-stack integration testing.

OBJECTIVES:

To connect the React frontend with Spring Boot backend APIs for all CRUD operations. To implement end-to-end user flows from registration to document analysis.

To handle success and failure states in the frontend with appropriate user feedback. To perform integration testing by verifying that frontend actions are reflected in the PostgreSQL database.

To demonstrate complete full-stack functionality of the VakilSahay application.

SOFTWARE AND TOOLS REQUIRED:

Spring Boot 3.2.3 (backend), React 18 + Vite (frontend), PostgreSQL 17 (database), Postman (API testing), IntelliJ IDEA, Node.js

THEORY:

1. Full Stack Integration

Full stack integration refers to connecting the frontend user interface with the backend API and database to form a complete working application. In VakilSahay, the React frontend communicates with the Spring Boot REST API via HTTP, which in turn performs operations on the PostgreSQL database. The complete data flow is: User Action (React UI) -> Axios HTTP Request -> Spring Boot Controller -> Service -> Repository -> PostgreSQL Database -> JSON Response -> React State Update -> UI Re-render.

2. CORS Configuration

Cross-Origin Resource Sharing (CORS) is a browser security mechanism that restricts HTTP requests made from one origin to another. Since the React frontend runs on port 5173 and the Spring Boot backend runs on port 8080, CORS must be explicitly configured. In VakilSahay, Spring Security is configured to allow requests from localhost:5173, localhost:3000, and 127.0.0.1:5173 with all required HTTP methods and headers.

3. Circular Dependency Resolution

Spring Boot's dependency injection can create circular dependencies when beans depend on each other. In VakilSahay, the SecurityConfig -> AuthService -> AuthenticationManager cycle was resolved by: (1) changing SecurityConfig to inject UserDetailsService interface instead of AuthService, (2) changing JwtAuthFilter to inject UserDetailsService instead of AuthService, and (3) annotating AuthenticationManager injection in AuthService with @Lazy so it is resolved at runtime rather than startup.

4. Integration Testing

Integration testing verifies that different parts of the application work together correctly. For VakilSahay, integration testing involves: uploading a document through the UI, waiting for the asynchronous analysis pipeline to complete, verifying that clauses appear in the clause viewer

with correct severity scores, confirming that the risk report shows the correct overall score, and checking

that deleting a document removes it from both the UI and the database.

FULL STACK DATA FLOW:

The following diagram describes the complete request-response lifecycle for document upload and analysis in VakilSahay:

Step 1	User drags a PDF file onto the UploadPage component in the React frontend.
Step 2	React calls documentsApi.upload(file) which sends a multipart POST request to /api/documents/upload with the JWT token in the Authorization header.
Step 3	JwtAuthFilter validates the token and sets the authenticated user in SecurityContext.
Step 4	DocumentController receives the file and calls DocumentService.uploadDocument().
Step 5	DocumentService saves the document record with status UPLOADED to PostgreSQL via DocumentRepository.
Step 6	DocumentService triggers analyzeAsync() as an @Async background task and returns the document ID (202 Accepted) to the React frontend.
Step 7	The async task extracts text using Apache Tika (DocumentParserService), detects document type, splits text into clauses, and runs each clause through ClauseAnalyzerService.
Step 8	ClauseAnalyzerService classifies each clause using the Indian Legal Clause Taxonomy, computes a severity score (0-100), generates a plain English explanation, and attaches an Indian law reference.
Step 9	Analyzed clauses and the AnalysisReport are saved to PostgreSQL. Document status is updated to COMPLETED.
Step 10	React polls GET /api/documents/{id} every 2 seconds. When status is COMPLETED, user is redirected to the DocumentPage.
Step 11	DocumentPage fetches clauses via GET /api/documents/{id}/clauses and renders each clause with severity badges, expandable details, law references, and recommendations.

CRUD OPERATIONS THROUGH UI:

Create — Upload New Document:

Navigate to <http://localhost:5173/upload>

Drag and drop a PDF, DOCX, or TXT file onto the upload zone.

Click Analyze this document. A progress bar shows upload completion.

The UI transitions to a processing spinner. Analysis runs in the background.

On completion, the user is redirected to the DocumentPage with full analysis. Database verification: `SELECT * FROM documents WHERE user_id = <id>;` — new record appears with status COMPLETED.

Read — View Document and Clauses:

On the Dashboard, all documents are listed with status badges and risk scores. Click any completed document to open DocumentPage.

The Clauses tab shows all clauses. Use filter buttons to show only CRITICAL, HIGH, MEDIUM, or LOW risk clauses.

Click any clause card to expand and view plain English explanation, Indian law reference, and recommendation.

The Full Report tab shows the overall risk score gauge, top 3 concerns, and recommendations.

Update — Re-Analyze Document:

On the DocumentPage, click the Re-analyze button in the top action bar.

The document status changes to ANALYZING and the analysis pipeline re-runs. After completion, refreshed clause data and updated risk scores are displayed. API call: `PUT /api/documents/{id}/reanalyze` returns 202 Accepted.

Delete — Remove Document:

On the Dashboard, click the trash icon next to any document.

A confirmation dialog appears. Click OK to confirm deletion.

The document is removed from the list immediately.

API call: `DELETE /api/documents/{id}` returns 204 No Content.

Database verification: All clauses and analysis report are cascade-deleted automatically.

ERROR AND SUCCESS HANDLING:

VakilSahay implements comprehensive error handling at all layers:

401 Unauthorized: If the JWT token expires, the Axios response interceptor clears auth state and redirects to /login automatically.

404 Not Found: If a document ID does not exist, the GlobalExceptionHandler returns `{status: 404, error: 'Document Not Found'}` and the React page shows a toast notification. **422 Unprocessable:** If an unsupported file type is uploaded, the error message from the backend is displayed as a toast.

Analysis Failed: If the async analysis pipeline throws an exception, the document status is set to FAILED and the UI shows a failed badge.

Success Toasts: Every successful operation (upload, delete, re-analyze) shows a green success notification using React Hot Toast.

INTEGRATION TEST RESULTS:

Test Scenario	Expected	Result
Register and login from UI	JWT stored, dashboard loads	PASS
Upload loan agreement .txt file	Analysis completes, clauses shown	PASS

View clause with sole discretion	Severity: CRITICAL, score > 75	PASS
Filter clauses by HIGH severity	Only HIGH clauses displayed	PASS
Full report tab shows risk gauge	Risk score and summary shown	PASS
Re-analyze document	Status resets to ANALYZING	PASS
Delete document from dashboard	Document removed from list and DB	PASS
Upload invalid file type (.exe)	Error toast: unsupported file type	PASS
Access dashboard without login	Redirected to /login	PASS
Token expiry simulation (401)	Auto logout and redirect to login	PASS

CONCLUSION:

The VakilSahay full stack application was successfully integrated with complete frontend-backend communication. The React frontend running on port 5173 communicates with the Spring Boot backend on port 8080 via Axios HTTP requests with automatic JWT token injection. All four CRUD operations — Create (document upload), Read (document list and clause viewer), Update (re-analysis), and Delete (document removal with cascade) — were verified to work correctly through the UI with the changes reflected in the PostgreSQL database. The application handles all error and success states gracefully with user-friendly toast notifications. A circular dependency issue in Spring Security configuration was resolved by injecting UserDetailsService interface instead of AuthService and using @Lazy injection for AuthenticationManager. All 10 integration test scenarios passed, confirming complete full-stack functionality of the VakilSahay legal document analyzer.
